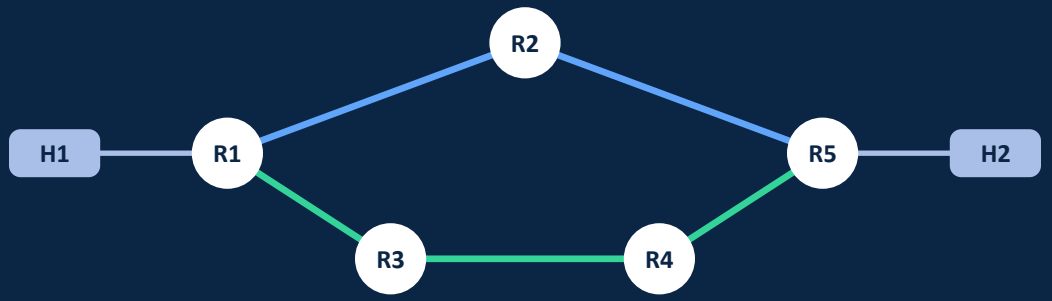
AIORI HACKATHON 3 • PS-020

CATEGORY – FUTURE 6G AND NETWORKING

Local SRv6 Path Steering and Measured Failure Recovery

An SRv6 path-steering prototype with scripted backup switching and recovery analysis



Team Code - **A3-PS020-TC231**

Team Name – Team Sage

1. **Abhinav G**
2. **Rajat R Pai**

Mentor - Chaitra Ma’am

Problem Description

1. Ordinary IPv6 forwards packets hop-by-hop along routes selected by routing protocols. A link failure can interrupt traffic while the network reconverges.

2. SRv6 allows an ingress router to steer packets along an explicit path by encoding an ordered list of Segment Identifiers (SIDs) in the IPv6 Segment Routing Header.

3. The challenge is to compare ordinary IPv6 routing with two explicit SRv6 paths and a scripted backup-policy switch using the same five-router topology, traffic load and controlled link failure.

4. The comparison must verify SID traversal using captures, measure packet loss and recovery timing, derive loss-equivalent interruption, and calculate encapsulation overhead cross-checked using ICMP captures.

Main question

Does our explicit SRv6 backup switch reduce disruption in this lab?

Comparison

Ordinary IPv6 baseline vs. scripted SRv6 path switch.

Evaluation metrics

Packet loss and recovery timing; calculated encapsulation overhead, cross-checked using ICMP captures.

TEST BED

5 routers R1–R5 | FRRouting + OSPFv3 underlay | Linux + Docker | R2-R5 link failure | UDP 1,000 pps

Background

IPv6

Ordinary IPv6 routing

Hop-by-hop forwarding uses the routing table. Link changes can trigger routing updates and reconvergence.

SR

Segment Routing

The ingress steers a packet through an ordered list of segments instead of only the shortest path.

SRv6

SRv6

Segments are IPv6 SIDs. The Segment Routing Header carries the list; Segments Left tracks progress.

Segment Routing Header (RFC 8754) — the fields we read in the captures

Next Header	Hdr Ext Len	Routing Type	Segments Left	Last Entry	Flags	Tag	Segment List
	`len=4 | len=6`	`type=4`	`segleft`	`last-entry`	`flags=0x0`	`tag=0`	`[0] … [n]  (reverse order)`

Why this matters for our experiment

1. Predefine a primary and a different backup path.
2. Inspect SID progression in packet captures, not just the route command.
3. Compare interruption around the same controlled link failure.
4. Extra header bytes are the cost, so recovery is not the only metric.

Literature & RFC Review

RFC 8402

Segment Routing Architecture

Defines SR as source routing through an ordered list of segments, over MPLS and IPv6, with the ingress/headend role.

PROJECT RELEVANCE

R1 is the headend that imposes the SID list.

RFC 8754

IPv6 Segment Routing Header

Defines the SRH: the SID list, the active destination and the Segments Left field.

PROJECT RELEVANCE

Read in captures: type=4, segleft, last-entry, SID list.

RFC 8986

SRv6 Network Programming

Defines the SRv6 network-programming model and endpoint behaviours for segment processing and decapsulation.

PROJECT RELEVANCE

Intermediate routers process SIDs; R5 decapsulates traffic before forwarding to H2.

How we used these standards

- **RFC 8402:** Provides the architectural basis for steering traffic through explicit primary and backup segment paths.
- **RFC 8754:** Defines the SRH fields used to verify SID traversal in packet captures and calculate header overhead.
- **RFC 8986:** Explains the SRv6 endpoint behaviours underlying segment processing and decapsulation.

Proposed Solution: Our Approach

1. Testbed and IPv6 baseline

We built five routers and two hosts using Linux, Docker and FRRouting. OSPFv3 supported the underlay; ordinary IPv6 provided the comparison baseline.

2. Primary and backup SRv6 paths

R1 encapsulated traffic with explicit SID lists. Policy A selects Path A (R1–R2–R5); Policy B selects Path B (R1–R3–R4–R5).

3. Controlled failure and monitoring

We disabled R2 eth2. R1 used a 5 ms missing-response threshold, which affects detection timing. The probe interval is unspecified here; 2 of 7 attempts (29%) triggered before the cut.

4. Scripted recovery

Our script activated Policy B over the existing backup path by replacing R1's two-SID route on eth2 with three SIDs on eth3. R5 decapsulated traffic.

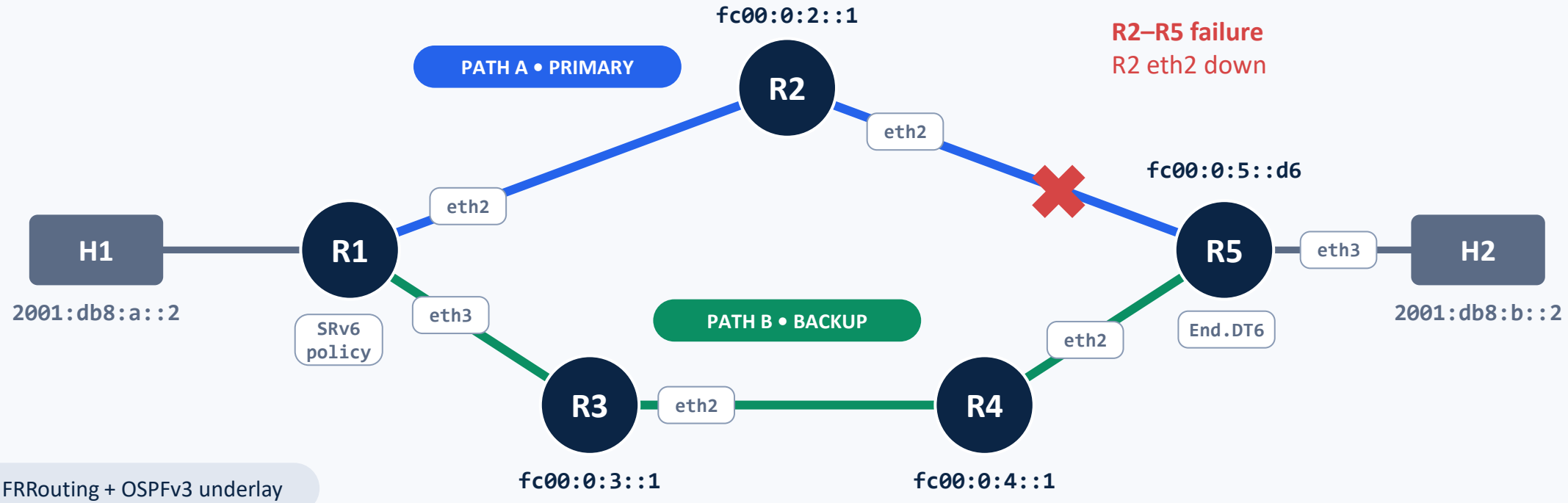
5. Path and recovery verification

ICMP captures verified SID progression and decapsulation. Route outputs and timestamped logs documented the policy change; recv.csv recorded UDP recovery separately.

6. Repeated comparison and analysis

We compared identical UDP traffic across five valid runs per configuration: 1,000 pps, 20 s, 64 B payload. Loss-equivalent interruption and encapsulation overhead were derived.

Our Proposed Solution: Topology and SID Paths



PATH A • PRIMARY • 2 SIDs

`fc00:0:2::1 → fc00:0:5::d6`

`dev eth2 • encap seg6`

PATH B • BACKUP • 3 SIDs

`fc00:0:3::1 → fc00:0:4::1 → fc00:0:5::d6`

`dev eth3 • encap seg6`

Implementation and Path Verification

PATH A

2 SIDs • ping 3/3, 0% loss

R1 • eth2 out
fc00:0:2::1
SL = 1

→

R2 • eth2 out
fc00:0:5::d6
SL = 0

→

R5 • eth3 out
SRH removed
plain ICMPv6

PATH B

3 SIDs • ping 3/3, 0% loss

R1 • eth3 out
fc00:0:3::1
SL = 2

→

R3 • eth2 out
fc00:0:4::1
SL = 1

→

R4 • eth2 out
fc00:0:5::d6
SL = 0

→

R5 • eth3 out
SRH removed
plain ICMPv6

Path A • raw tcpdump at R2 eth2 (last SID hop)

```
===== POLICY A: r2 eth2 OUTGOING =====
reading from file /tmp/verified_A_r2_eth2.pcap, link-type EN10MB (Ethernet), snapshot length 262144
14:33:36.768329 IP6 (flowlabel 0x79e2e, hlim 62, next-header Routing (43) payload length: 144) 2001:db8:12::1 > fc00:0:5::d6: RT6 (len=4, type=4, segleft=0, last-ent
ry=1, flags=0x0, tag=0, [0]fc00:0:5::d6, [1]fc00:0:2::1) IP6 (flowlabel 0x79e2e, hlim 64, next-header ICMPv6 (58) payload length: 64) 2001:db8:a::2 > 2001:db8:b::2:
[icmp6 sum ok] ICMP6, echo request, id 571, seq 0
14:33:37.768503 IP6 (flowlabel 0x79e2e, hlim 62, next-header Routing (43) payload length: 144) 2001:db8:12::1 > fc00:0:5::d6: RT6 (len=4, type=4, segleft=0, last-ent
ry=1, flags=0x0, tag=0, [0]fc00:0:5::d6, [1]fc00:0:2::1) IP6 (flowlabel 0x79e2e, hlim 64, next-header ICMPv6 (58) payload length: 64) 2001:db8:a::2 > 2001:db8:b::2:
[icmp6 sum ok] ICMP6, echo request, id 571, seq 1
14:33:38.768727 IP6 (flowlabel 0x79e2e, hlim 62, next-header Routing (43) payload length: 144) 2001:db8:12::1 > fc00:0:5::d6: RT6 (len=4, type=4, segleft=0, last-ent
ry=1, flags=0x0, tag=0, [0]fc00:0:5::d6, [1]fc00:0:2::1) IP6 (flowlabel 0x79e2e, hlim 64, next-header ICMPv6 (58) payload length: 64) 2001:db8:a::2 > 2001:db8:b::2:
[icmp6 sum ok] ICMP6, echo request, id 571, seq 2
```

Path B • raw tcpdump at R4 eth2 (last SID hop)

```
===== POLICY B: r4 eth2 OUTGOING =====
reading from file /tmp/verified_B_r4_eth2.pcap, link-type EN10MB (Ethernet), snapshot length 262144
14:33:44.357282 IP6 (flowlabel 0x79e2e, hlim 61, next-header Routing (43) payload length: 160) 2001:db8:13::1 > fc00:0:5::d6: RT6 (len=6, type=4, segleft=0, last-ent
ry=2, flags=0x0, tag=0, [0]fc00:0:5::d6, [1]fc00:0:4::1, [2]fc00:0:3::1) IP6 (flowlabel 0x79e2e, hlim 64, next-header ICMPv6 (58) payload length: 64) 2001:db8:a::2 >
 2001:db8:b::2: [icmp6 sum ok] ICMP6, echo request, id 577, seq 0
14:33:45.402847 IP6 (flowlabel 0x79e2e, hlim 61, next-header Routing (43) payload length: 160) 2001:db8:13::1 > fc00:0:5::d6: RT6 (len=6, type=4, segleft=0, last-ent
ry=2, flags=0x0, tag=0, [0]fc00:0:5::d6, [1]fc00:0:4::1, [2]fc00:0:3::1) IP6 (flowlabel 0x79e2e, hlim 64, next-header ICMPv6 (58) payload length: 64) 2001:db8:a::2 >
 2001:db8:b::2: [icmp6 sum ok] ICMP6, echo request, id 577, seq 1
14:33:46.404868 IP6 (flowlabel 0x79e2e, hlim 61, next-header Routing (43) payload length: 160) 2001:db8:13::1 > fc00:0:5::d6: RT6 (len=6, type=4, segleft=0, last-ent
ry=2, flags=0x0, tag=0, [0]fc00:0:5::d6, [1]fc00:0:4::1, [2]fc00:0:3::1) IP6 (flowlabel 0x79e2e, hlim 64, next-header ICMPv6 (58) payload length: 64) 2001:db8:a::2 >
 2001:db8:b::2: [icmp6 sum ok] ICMP6, echo request, id 577, seq 2
```

Optimization Proposed and Measurement Method

Traffic

H1 → H2 UDP • 1,000 pps • 20 s • 20,000 intended packets • 64 B payload. Identical for both cases.

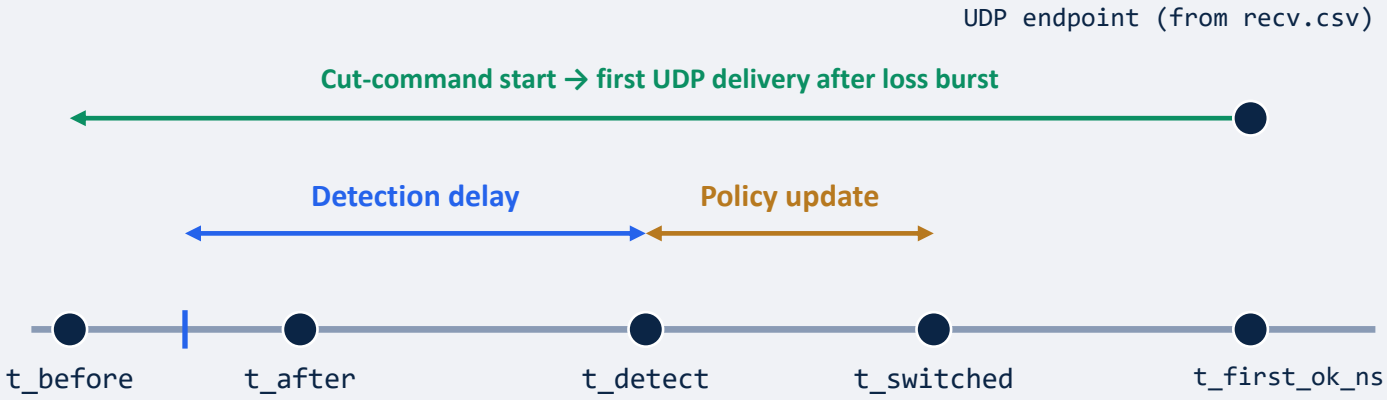
Baseline

Final baseline: runs 6–10 (5 reported). Runs 1–5 were development runs and are not reported.

SRv6

5 valid / 7 attempts; 2 premature triggers (29%). R1 used a 5 ms threshold; detection depends on it. Probe interval is unspecified here.

Timing definitions used in our analysis



Run 7: UDP recovery 7.89 ms; probe recovery 7.94 ms (UDP ≈0.05 ms earlier).

Loss-equivalent interruption = lost packets × 1 ms at 1,000 pps; not a receiver-gap measurement.

Calculated SRv6 overhead

+80 B

Path A (2 SIDs)

40 B IPv6 + 40 B SRH

+96 B

Path B (3 SIDs)

40 B IPv6 + 56 B SRH

```
SRH = 8 B fixed + 16 B × SIDs
40 B = 8 + 2×16     56 B = 8 + 3×16
```

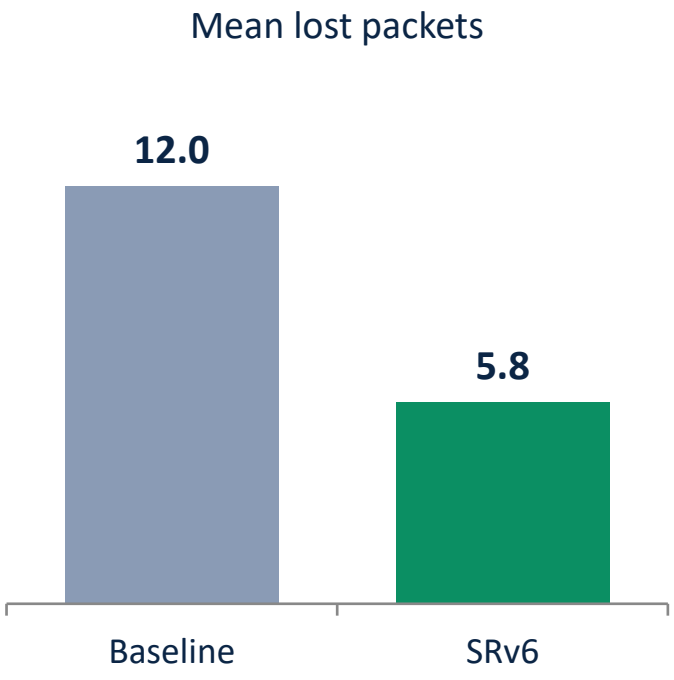
Calculated for the 64 B UDP-payload experiment.

Measured Results

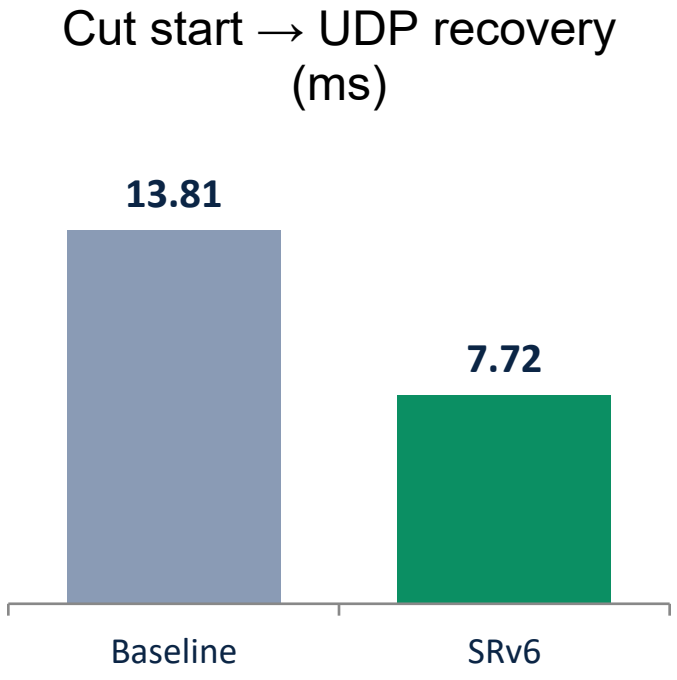
Metric	Baseline (ordinary IPv6)	SRv6 (scripted switch)
Valid runs	5 final runs (6–10)	**5 valid / 7 attempts**
Mean lost packets	12.0 ± 1.00	**5.8 ± 0.45**
Packet loss rate	0.060%	**0.029%**
Derived interruption (loss × 1 ms)	12.0 ± 1.00 ms	**5.8 ± 0.45 ms**
Cut start → first UDP after loss	13.81 ± 1.03 ms	**7.72 ± 0.43 ms**
Detection from estimated midpoint	*N/A – routing protocol recovery*	**4.89 ± 0.32 ms**
Policy-update duration	*N/A – no scripted policy update*	**1.72 ± 0.22 ms**

± = sample SD over five valid runs. Detection uses the estimated cut midpoint (Run 7 cut interval: 1.45 ms). Policy-update duration excludes later delivery. Baseline recovery mechanism was not isolated; the 12–14 ms result does not establish OSPFv3 reconvergence time.

What the Results Show



12.0 ± 1.00 → 5.8 ± 0.45



13.81 ± 1.03 → 7.72 ± 0.43

51.7% lower

Mean loss: 12.0 → 5.8 packets

44.1% lower

Cut start → UDP recovery: 13.81 → 7.72 ms

Run 7 • results/srv6/run7

```
Run 7: 6 packets lost
Derived interruption: 6.0 ms
Detection: 5.30 ms
Policy update: 1.77 ms
UDP recovery: 7.89 ms (recv.csv)
Probe recovery: 7.94 ms (fail.txt)

t_first_ok_ns is the probe timestamp.
UDP recovery uses receiver records.
```

Five valid runs per configuration. Baseline recovery mechanism was checked for this

Solution Architecture & Design

DATA PLANE

R1 activates Policy B: 3 SIDs, eth3

Failure injection: R2 eth2 down

H1 sender
UDP traffic

→

R1 ingress
Primary SRv6 policy

→

5-router underlay
OSPFv3 / FRR

→

H2 receiver
Delivery resumes

CONTROL / EVIDENCE PATH (steps 1 → 4)

1 Cut script
Takes R2 eth2 down
`t_before_ns / t_after_ns`

→

2 Failure detector
Missing probe responses (5 ms)
`t_detect_ns`

→

3 Policy update
Installs 3-SID backup list on R1
`t_switched_ns`

→

4 Logs & captures
Probe success + UDP receiver logs
`t_first_ok_ns`

Run 7 screenshot — original evidence

```
1. POLICY BEFORE FAILURE
2001:db8:b::2  encap seg6 mode encap segs 2 [ fc00:0:2::1 fc00:0:5::d6 ] dev eth2 metric 1024 pref medium

2. CONTROLLED R2 eth2 LINK-DOWN COMMAND TIMESTAMPS
t_before_ns=1791810453385326911
t_after_ns=1791810453386772712

3. FAILURE DETECTION AND POLICY-SWITCH TIMESTAMPS
t_detect_ns=1791810453391349580
t_switched_ns=1791810453393115574
t_first_ok_ns=1791810453393270805
probes_sent=21968

4. POLICY AFTER FAILURE
2001:db8:b::2  encap seg6 mode encap segs 3 [ fc00:0:3::1 fc00:0:4::1 fc00:0:5::d6 ] dev eth3 metric 1024 pref medium

5. MEASURED PACKET LOSS AND RECOVERY
results/srv6/run7: VALID
   lost@cut=6  interruption=6.0 ms (resolution 1.0 ms)  detection delay=5.3 ms  switch cmd=1.8 ms  first delivery after cut=7.9 ms

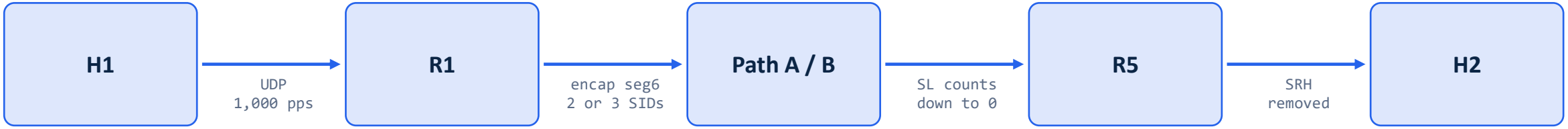
1/1 runs valid
  packets lost (n=1): 6.00
  interruption ms (n=1): 6.00
  detection delay ms (n=1): 5.30
  switch cmd ms (n=1): 1.77
  first delivery after cut ms (n=1): 7.89
abhinav@Abhinav:~/srv6$
```

Readable policy change and timestamps

```
Before: segs 2 [fc00:0:2::1 fc00:0:5::d6] dev
eth2
After: segs 3 [fc00:0:3::1 fc00:0:4::1
fc00:0:5::d6] dev eth3
t_before_ns=1791810453385326911
t_after_ns=1791810453386772712
t_detect_ns=1791810453391349580
t_switched_ns=1791810453393115574
t_first_ok_ns=1791810453393270805 (probe)
UDP recovery: 7.89 ms from recv.csv; probe:
7.94 ms.
```

Data Flow Diagram

PACKET PATH



MEASUREMENT / CONTROL PATH



SAVED EVIDENCE

- sender / receiver logs
- cut logs
- before / after routes
- packet captures
- per-run analysis

Evidence: fail.txt (probe) + recv.csv (UDP) + routes + verified_20261012_143331 captures

Deployment Timeline and Project Milestones

4 Oct	**Problem Analysis and Solution Planning**	Reviewed PS-020 and defined the topology, failure scenario and comparison metrics.
6 Oct	**Testbed and IPv6 Baseline Setup**	Established the five-router network with OSPFv3 and ordinary IPv6 routing.
6 Oct	**SRv6 Path Configuration**	Configured explicit primary and backup SID paths at R1.
8 Oct	**Failure Detection and Backup Switching**	Implemented the monitoring script and automated backup-policy update.
11 Oct	**Comparative Testing**	Repeated IPv6 and SRv6 failure tests under identical traffic conditions.
12 Oct	**Path Verification and Evidence Collection**	Verified SID traversal and collected captures, route outputs and timing logs.
12–13 Oct	**Submission Preparation**	Prepared the presentation, repository and team introduction video.
14 Oct	**Level 1 Submission**	Complete and submit Steps 1–7.

NEXT PHASE • 16–31 OCTOBER • SUBJECT TO QUALIFICATION

Refine failure detection, measure interruption directly from receiver logs, and complete mentoring and Steps 8–10.

Bibliography

1. C. Filsfils et al., RFC 8402, “Segment Routing Architecture,” IETF, July 2018.
2. C. Filsfils et al., RFC 8754, “IPv6 Segment Routing Header (SRH),” IETF, March 2020.
3. C. Filsfils et al., RFC 8986, “Segment Routing over IPv6 (SRv6) Network Programming,” IETF, February 2021.
4. AIORI Hackathon 3, PS-020: “Local SRv6 Path Steering and Measured Failure Recovery,” supplied problem statement.

What we can claim from our prototype

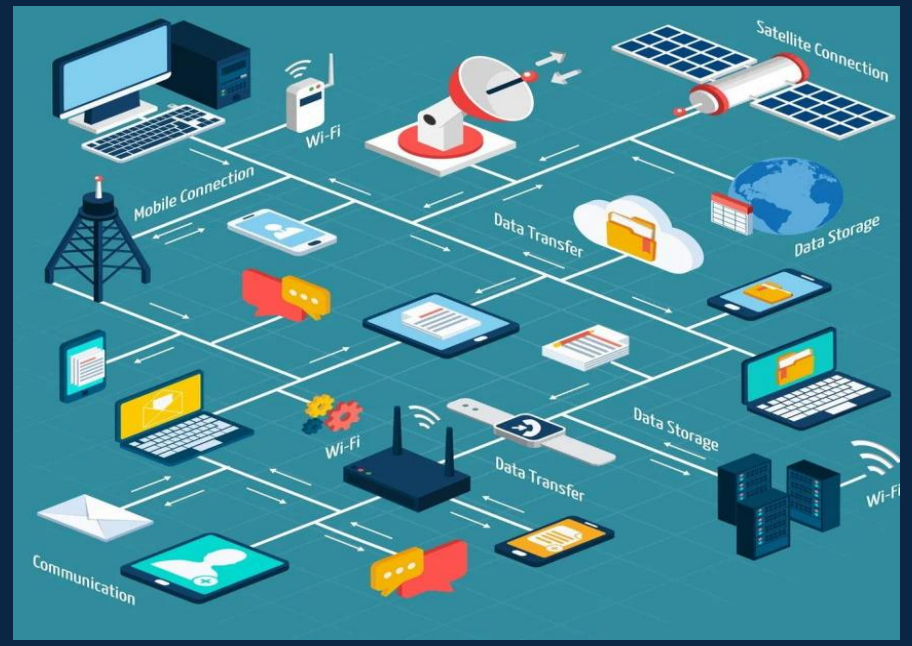
- We implemented and verified two explicit SRv6 paths with a scripted backup-policy switch.
- Five valid SRv6 runs showed lower average loss and shorter UDP recovery than five final IPv6 baseline runs.
- Two SRv6 attempts triggered before the cut and were excluded. The trigger mechanism requires refinement.
- Results are limited to this testbed and sample; the baseline recovery mechanism was not isolated.

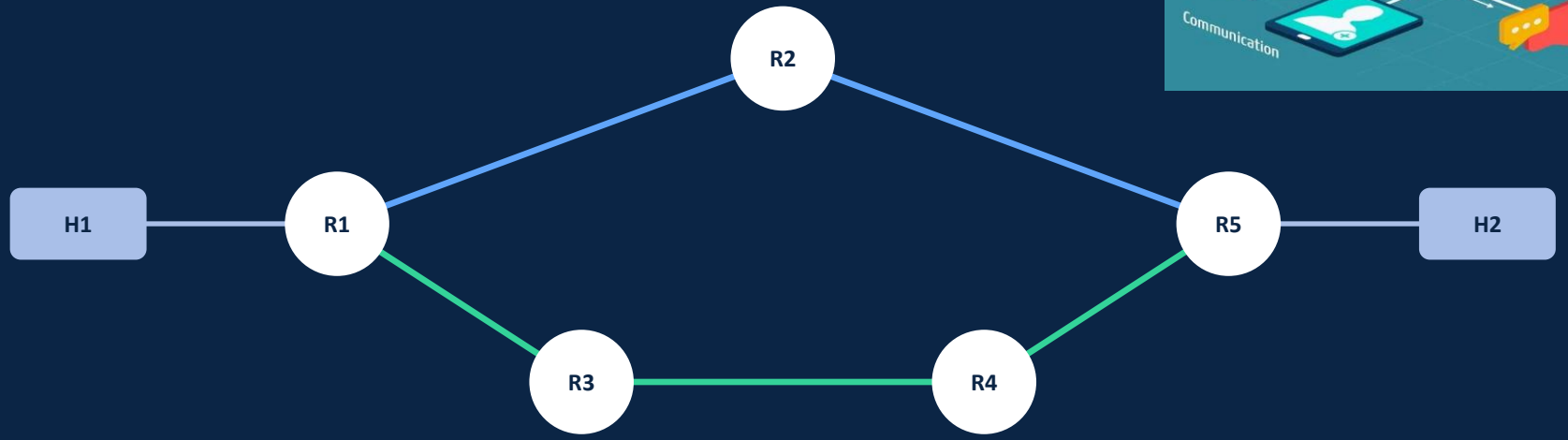
Next steps

- Investigate premature probe triggers
- Measure receiver interruption directly
- Verify baseline route-change mechanism

Thank You for Your Attention!

PS-020 • Local SRv6 Path Steering and Measured Failure Recovery





Abhinav G • Rajat R Pai • A3-PS020-TC231 –Team Sage